# laravel-workflow 2.0

A step-by-step guide to adding this Claude Code workflow to your Laravel app, and what it does once it's in.


1. What is laravel-workflow?
2. What you need first
3. Installing it, step by step
4. Laravel Boost (recommended)
5. Your first task: plan, approve, build
6. Running many tasks safely
7. All commands and helpers
8. The specialist agents
9. Automatic checks (hooks)
10. Post and Page content: Gutenberg
11. Redis, uploads, S3 and CDN
12. Settings you can change
13. A typical day
14. Problems and fixes


## 1. What is laravel-workflow?

laravel-workflow is a **kit for Claude Code**, not a Composer package. Nothing is added to your app's code. It copies instructions and checks into your project's `.claude/` folder, and they change how Claude Code works on your Laravel project.

- **It plans before it builds.** Opus turns your request into small task files with clear "done" checks. You read and approve them.
- **Sonnet builds each task,** directly for small tasks and with specialist agents for large ones.
- **A second agent checks the work.** It sees only the task and the changes, never the builder's reasoning.
- **Automatic checks** catch the mistakes that only show up in production: `env()` outside config, mass assignment, raw SQL, a missing `@csrf`, N+1 queries, and more.
- **It knows Laravel 13** (rules checked against laravel.com), and it uses **Laravel Boost** when you have it.
- **Post and Page modules always use the Gutenberg block editor,** never CKEditor.
- **It's safe to undo:** you can get one commit per task and roll back any single task.

> **In one sentence**
> Describe what you want with `/laravel:write-task`, approve the plan, run `/laravel:task`, and every task is built, independently verified and recorded.

## 2. What you need first

| Requirement | Why |
|---|---|
| A Laravel 11, 12 or 13 app | It needs `artisan` and `laravel/framework` in `composer.json`. The kit detects the version. |
| Claude Code, bash, PHP, `jq` | The checks are shell scripts; `jq` merges settings safely. |
| git (recommended) | Enables task diffs, per-task commits and undo. |

| | |
|---|---|
| Pint, Pest/PHPUnit, PHPStan (optional) | Used if present; nothing is installed for you. |
| Playwright (optional) | Lets the checker verify `[browser]` criteria in a real browser. |

## 3. Installing it, step by step

### 1 Install into your app

```
cd ~/www/hobbies/laravel-workflow
./install.sh /path/to/your-laravel-app        # add --dry-run to preview
```

This copies the kit into `.claude/` and merges `settings.json`. It backs up the old file, replaces older kit entries, and removes permission rules that earlier versions shipped and later removed. If PHPStan is installed, it also records the existing static-analysis findings, so Claude is only held to new ones.

### 2 Open Claude Code in your app

```
cd /path/to/your-laravel-app
claude
```

You'll see a short report with the Laravel and PHP versions, the view layer, drivers, storage, the editor rule and the verify commands.

### 3 Run setup once

```
/laravel:setup
```

It creates `tasks/`, fills `.claude/project-context.md` with facts about your repo, adds a short pointer to `CLAUDE.md`, recommends Laravel Boost if it's missing, and tells you what won't work here.

> **Commit**
> `.claude/`, `CLAUDE.md` and `tasks/` so your team gets the same workflow.

## 4. Laravel Boost (recommended)

Laravel Boost is Laravel's official MCP server. With it installed, the kit's agents can use its **read-only** tools:

| Boost tool | Used for |
|---|---|
| `search-docs` | Documentation for your exact package versions, checked before relying on memory |
| `database-schema`, `database-query` | Reading the real schema and stored data (read-only) instead of guessing |
| `last-error`, `read-log-entries`, `browser-logs` | Seeing the actual exception when something fails |

| | |
|---|---|
| `record-rule` | Team-shared project rules in `.ai/rules`, used by the main session only, never by agents |

```
composer require laravel/boost --dev
php artisan boost:install          # choose Claude Code: guidelines, skills, MCP
```

Claude Code asks once to approve the project's `.mcp.json` server.

## 5. Your first task: plan, approve, build

### 1 Describe what you want

```
/laravel:write-task an admin screen where editors manage their own news and can
delete one
```

The planner (Opus) asks questions if something is unclear, splits the work, sets the order and size, and writes **drafts**. It never writes code.

### 2 Read the drafts, then approve

```
- [ ] [test]    Deleting another user's news returns 403 and deletes nothing
- [ ] [browser] Guests are redirected to "/"
- [ ] [review]  Authorization lives in a NewsPolicy
```

Each criterion is tagged:

- `[test]`: a Pest/PHPUnit test must prove it.
- `[browser]`: it has to be seen in the rendered app.
- `[review]`: reading the code is enough.

```
bash .claude/hooks/tasks.sh approve all
```

### 3 Build it

```
/laravel:task
```

Sonnet builds the task, runs the checks, hands the task and its diff to `laravel-verifier`, fixes anything that fails (two rounds at most, then it marks the task blocked), and closes it.

## 6. Running many tasks safely

```
bash .claude/hooks/run-tasks.sh --checkpoint          # fresh session per task, one
commit each
```

- **Checkpoint commits** go on a new `nw/batch-…` branch. Each holds only that task's files: never `.env` or keys, never edits that were already there.

- **Undo one task:** `tasks.sh undo 004`. Add `--cascade` if later tasks depend on it.
- **Batch limits:** `/laravel:task all` stops after 3 tasks, because quality drops as context grows. Use the runner for more.

## 7. All commands and helpers

| Command | Model | Use it to… |
|---|---|---|
| `/laravel:setup` | current | Connect the workflow (once) |
| `/laravel:write-task <what>` | Opus | Plan work into draft tasks |
| `/laravel:task [id\|all\|list]` | Sonnet | Build tasks: your main command |
| `/laravel:feature <id>` | Sonnet | Full phased build: migration → model → route → view → tests → review |
| `/laravel:test`, `/laravel:review`, `/laravel:audit`, `/laravel:ship` | current / Opus reviewer | Tests, review, audit, and the pre-ship checks |

**Terminal helpers**, all in `bash .claude/hooks/tasks.sh …`:

- `list`: show every task and its status.
- `approve`: approve drafts.
- `diff`: show everything a task changed.
- `undo`: revert a task's checkpoint commit.
- `stats`: time per task, batch cost, the checker's first-pass rate, and which checks fire most.

## 8. The specialist agents

| Agent | Job | Can edit? |
|---|---|---|
| `laravel-route-builder` | Routes, FormRequests, controllers, middleware, policies | Yes |
| `laravel-data-layer` | Migrations, models, queries, services, caching, jobs | Yes |
| `blade-ui-builder` | Views, components, Alpine, styling, the block-editor island | Yes |
| `laravel-test-writer` | Pest/PHPUnit tests | Yes |

| | | |
|---|---|---|
| `laravel-verifier` (new) | Checks every "done" criterion against the task and diff | Throwaway browser test files only |
| `laravel-reviewer` | Security and correctness review (always Opus) | No |
| `laravel-perf-auditor`, `blade-a11y-auditor` | Performance; accessibility | No |

## 9. Automatic checks (hooks)

| Check | When | What it does |
|---|---|---|
| `guard-php` | after edits | Blocks:<br>• `env()` outside config;<br>• mass assignment from the raw request;<br>• raw SQL built from variables;<br>• `dd()`;<br>• Laravel 8/9 file structure;<br>• SSRF and path traversal from request input;<br>• `upsert()` without its unique columns (Laravel 13).<br>Advises on N+1 queries, Redis `KEYS`, `Cache::flush()`, unsafe upload names, image work done inside the request, and the Laravel 13 changes. |
| `guard-blade` | after edits | Blocks a missing `@csrf`, unescaped request data, and queries in views. |
| `guard-styles` | after edits | Checks Tailwind classes against the version you actually have installed. |
| `guard-editor` | after edits | Flags CKEditor/TinyMCE/Summernote/Quill in Post/Page code. |
| `bash-guard`, `guard-secrets` | around shell commands | Files written by the shell get the same checks; reading secrets is blocked. |
| `verify` | before Claude finishes | Runs `php -l` on changed PHP; optionally Pint and PHPStan (only new findings fail). |

## 10. Post and Page content: Gutenberg

Every Post, Page, News or Article module uses **WordPress's own block editor** (`@wordpress/block-editor`), mounted as a small React component in your Blade form through Vite. It never uses CKEditor or TinyMCE.

- **Storage:** `content` holds the block format (exactly like WordPress); `content_html` is a **sanitized** copy made when you save.
- **Display:** the public page outputs `{!! $post->content_html !!}`, the only unescaped output allowed.

- **Images:** they upload through a protected Laravel route to your disk or CDN.
- **Tests:** the planner adds them automatically, covering harmful HTML being stripped, authors not editing each other's posts, and the editor saving a heading, a paragraph and an image.

Why not Laraberg? Its last release was in January 2023, and it relies on React 17, jQuery and Moment.

## 11. Redis, uploads, S3 and CDN

- **Redis:** phpredis is preferred (predis if your project already uses it). Cache data lives on its own connection. Every cached value needs a way to be cleared. Use tags only on Redis or Memcached. Never use `KEYS` or `Cache::flush()` on a shared Redis. On Laravel 13, cache arrays, not models.
- **Uploads:** validate with `File::image()->max('5mb')->dimensions(…)` (SVG is rejected by default). Store with random names, never the client's file name.
- **Public vs private:** public files use `storePublicly` and are served from the CDN. Private files are stored privately and served with `temporaryUrl()` after a permission check.
- **CDN:** set `AWS_URL=https://cdn.example.com` and `Storage::url()` returns CDN links. Store paths in the database, never full URLs. Never-changing file names with long cache headers mean you never have to purge the CDN.
- **Images:** resize, convert to WebP and create size variants in a **queued job**, not during the request.

## 12. Settings you can change

| Variable | Effect |
|---|---|
| `LARAVEL_WF_CHECKPOINT=commit` | One local commit per finished task |
| `LARAVEL_WF_DEV_URL=http://localhost:8000` | Lets `[browser]` checks run in a real browser |
| `LARAVEL_WF_VERIFY=syntax,lint,analyse` | What runs before Claude finishes |
| `LARAVEL_WF_PHP_GUARD` / `BLADE_GUARD` = `warn\|off` | Soften a guard |
| `LARAVEL_WF_EDITOR_GUARD`, `STYLE_GUARD`, `SECRETS_GUARD` = `off` | Turn a guard off |

**Update:** run `./install.sh /path/to/app` again. **Remove:** `./install.sh /path/to/app --uninstall` (your tasks and memory are kept).

## 13. A typical day

1. `/laravel:write-task …`, then answer its questions.
2. Read the drafts, then `tasks.sh approve all`.
3. Optional: start `php artisan serve` and set `LARAVEL_WF_DEV_URL`.
4. `bash .claude/hooks/run-tasks.sh --checkpoint`.
5. Check the result; undo any task you don't like.

6. `/laravel:review`, then `/laravel:ship`. When everything is green, merge the batch branch and push it yourself.

## 14. Problems and fixes

| Problem | Fix |
|---|---|
| Every turn fails on old PHPStan findings | `! bash .claude/hooks/verify.sh --baseline` |
| "task is still a draft" | `tasks.sh approve <id>` |
| A plan file in `tasks/` is ignored | That's intended: only files starting with `---` frontmatter are tasks. |
| "checkpoint: SKIPPED — secret files changed" | Add the file to `.gitignore`, then commit the task's files yourself. |
| `[browser]` items marked UNVERIFIED | Start the app, set `LARAVEL_WF_DEV_URL`, and install Playwright. |
| Boost tools not used | Install Boost (section 4) and approve the `.mcp.json` server. |
| A guard flags code you need | Set that guard to `warn` or `off` (section 12). |